

Function Calling 与 Tool Use

让模型提出动作，让 Runtime 安全执行

模型只负责「提议」，Runtime 才负责「执行」。这条边界守住了，你的 Agent 才能从「玩具」走向「生产级」。

本讲主线与学习目标

我们会先从 **Function Calling 的工作机制** 讲起，看模型怎样提出 Tool Call，Tool Call 为什么只是「候选动作」而不是「执行许可」；然后，我们会花大力气设计一套完整的 Tool Runtime，包括工具定义、**Input/Output/Error Schema**、**权限分级**、**超时重试**、**审计 Trace**；最后，我们再接入 MCP，把外部能力标准化地挂载到 Agent 上。

整个过程中，你会反复看到一条主线：模型只负责「提议」，Runtime 才负责「执行」。这条边界守住了，你的 Agent 才能从「玩具」走向「生产级」。

学完这一章，你将能自己动手实现一个订单查询和取消的工具，并把它跑在一个可治理、可追踪的运行上。

学习目标

完成本节后，学员应当能够：

- 1 说明 Function Calling 从模型生成工具调用到结果回传的完整工作机制；
- 2 为工具定义清晰的名称、描述、参数 Schema、返回 Schema、错误 Schema、权限等级和风险等级；
- 3 使用 Pydantic 分别定义并校验工具的输入、输出和错误协议；
- 4 解释模型怎样选择工具，并实现一次「模型决策—工具执行—结果回传—模型继续回答」的最小循环；
- 5 在 Tool Runtime 中统一处理参数校验、权限、审计、超时、重试、失败恢复和 Trace。

本节主线

模型提出 Tool Call



Runtime 校验、授权并执行



结构化 Tool Result 返回模型



模型继续决策或生成答案

其中，模型擅长理解用户意图和生成候选参数；Agent 开发工程师要亲自完成的，是工具协议、执行边界、权限规则、失败策略和运行记录。

模型已经能回答问题，为什么还需要 Tool Use?

核心结论：Function Calling 不是让模型获得执行权限，而是让模型用结构化方式提出「我想调用哪个工具、参数是什么」。真正的执行必须交给 Tool Runtime。

职责边界

角色	负责什么	不能替谁决定什么
模型	理解请求、选择候选工具、生成候选参数、阅读工具结果	不能决定自己是否有权限，也不能直接执行 Python 函数
工具函数	完成一项确定的业务能力	不能相信调用参数天然正确，也不负责整个 Agent Loop
Tool Runtime	查找工具、校验、鉴权、超时、重试、审计和结果归一化	不能替业务方定义权限和风险规则
Agent Loop / Harness	保存消息和状态，把 Tool Result 送回模型，决定继续或结束	不应绕过 Runtime 直接调用业务函数

Function Calling 工作机制

为什么模型返回了 Tool Call，程序还不能直接执行？

一次完整调用通常包含六步：

- 1 应用把用户消息和当前可用的工具 Schema 一起发送给模型；
- 2 模型决定直接回答，或者返回一个或多个 Tool Call；
- 3 每个 Tool Call 携带调用 ID、工具名称和 JSON 参数；
- 4 Tool Runtime 校验工具、参数、权限和风险，再执行真实函数；
- 5 Runtime 把成功结果或结构化错误作为 Tool Result 写回消息历史；
- 6 模型读取结果，继续调用工具、向用户追问，或者生成最终答案。

这条链路可以压缩成

User Message



LLM：直接回答，或生成 Tool Call



Tool Runtime：validate → authorize → execute → observe



Tool Result：必须带回原来的 tool_call_id



LLM：继续决策或完成回答

三类 ID 的职责要分开

ID	标识什么	生命周期
trace_id	一次 Agent 任务或请求	贯穿多轮模型调用和多次工具执行
tool_call_id	模型在某一轮提出的一次具体工具调用	从 Tool Call 产生，到对应 Tool Result 写回
order_id 等业务 ID	工具正在操作的真实业务对象	由业务系统定义，可能跨越很多 Agent 任务

tool_call_id 至少解决四个问题

- 第 1 解决消息协议的对应关系。Assistant Message 提出了哪些 Tool Call，后续就要为每个调用写回对应的 Tool Result。结果不能只依赖数组位置，也不能只依赖工具名称。
- 第 2 解除生成顺序与完成顺序的耦合。只读工具可以并行执行，谁先完成就先产生执行事件；最终无论怎样排序，都能通过调用 ID 找回来源。
- 第 3 区分成功和失败的恢复对象。同一批调用中，一个可能成功，一个可能参数错误，另一个可能超时。Agent Loop 必须知道应该修正或重试哪一个调用，不能把整批结果统一当成成功或失败。
- 第 4 建立 Trace 的父子关系。日志可以使用 trace_id 找到整个任务，再使用 tool_call_id 定位某次工具调用的参数摘要、权限判断、重试次数、耗时和最终结果。

工具定义：名称、描述、Schema、权限与风险

为什么不能把一个普通 Python 函数直接暴露给模型？

每个工具至少要有以下信息：

字段	解决的问题	示例
name	稳定、唯一地标识工具	search_orders
description	告诉模型何时使用、边界是什么	查询当前用户的订单，不修改订单
input_model	调用需要哪些参数	状态、开始日期、数量上限
output_model	成功后返回什么事实	订单列表、总数
error_model	失败后怎样分类和恢复	参数错误、拒绝、超时、上游失败
permission	调用者需要什么权限	order:read
risk	调用前需要怎样的控制	low、medium、high
timeout_seconds	最多等待多久	5 秒
max_retries	临时失败最多重试几次	2 次
idempotent	重复执行是否会产生额外副作用	查询为 True，取消订单通常为 False

建立 Harness 的工具单一事实源

- 1 · 注册阶段：ToolDefinition 进入 Tool Registry
- 2 · 模型阶段：to_model_tool() 只投影模型可见协议
- 3 · 执行阶段：Runtime 找回完整定义并执行治理链
- 4 · Loop 阶段：只接收校验后的 Output 或结构化 Error

Input Schema、Output Schema 与 Error Schema

把工具边界拆成三份 Schema

Schema	保护哪一侧	核心问题
Input Schema	模型到 Runtime	参数是否完整、类型是否正确、范围是否合理
Output Schema	业务系统到 Agent	成功结果是否稳定、后续 Loop 能否直接消费
Error Schema	失败到恢复逻辑	失败属于哪一类、能否重试、下一步是什么

三份 Schema 分别卡在不同位置：**Input 位于 Tool Call 与 handler 之间**；**Output 位于 handler 与 Tool Result 之间**；**Error 位于失败与下一轮决策之间**。它们不是三个孤立的数据类，而是 Harness 控制流的三个关口。

输入 Schema 设计中的关键原则：**模型只填写完成任务所需的业务参数**；身份和权限由 Runtime 从可信上下文注入。

工具选择： 模型怎样选择工具并生成参数？

模型是怎样知道该调用 `search_orders`，而不是直接回答？

工具选择质量主要受四个因素影响：

因素	工程师需要控制什么
工具集合	当前请求真正允许模型看到哪些工具
名称与描述	每个工具的用途、适用条件和禁止边界
参数 Schema	字段含义、枚举、必填项和范围
上下文	用户目标、已获得的 Tool Result 和当前状态

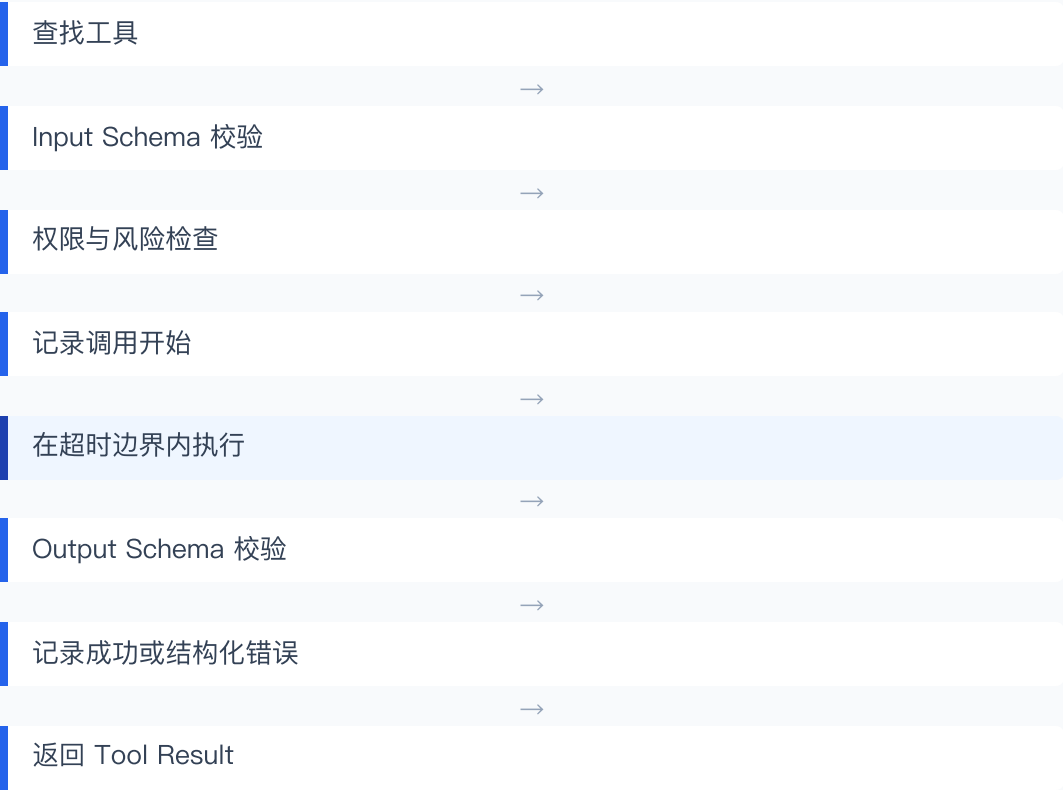
不要把所有平台工具都暴露给每一轮模型。权限过滤得越晚，模型越可能选择一个最后必然被拒绝的工具。更好的做法是：**Runtime 先根据用户、租户、Agent 角色和当前模式筛出可见工具，再把这份最小集合传给模型；真正执行前仍要再次鉴权。**

工具治理：校验、权限、审计、超时、重试、恢复与 Trace

为什么「工具成功调用一次」不等于 Tool Runtime 已经完成？

生产环境中的失败不只来自模型。这一节先给出统一执行顺序：

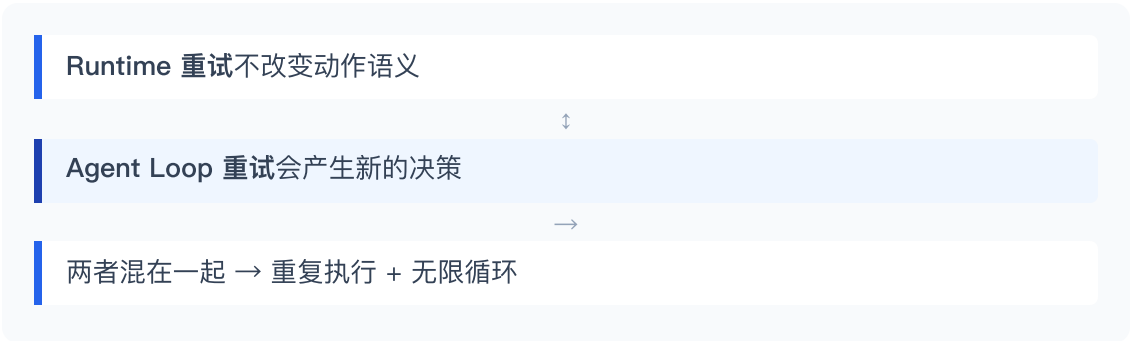
统一执行顺序



区分 Runtime 重试和 Agent Loop 重试

重试位置	适合处理什么	示例
Tool Runtime	同一个合法调用的瞬时基础设施故障	幂等查询遇到 502，原参数退避重试
Agent Loop	需要模型重新决策或修改参数	订单 ID 不存在，模型先查询候选订单

关键区分



DeepSeek Harness 的工具插件化设计

2.6.1 DeepSeek Harness 是什么？

DeepSeek Harness 是 DeepSeek AI 开源的一套 Agent Harness，目前处于 developer preview。它不是一个新模型，也不是 Function Calling API，而是运行 Agent 的工程框架。

可以先把 Harness 理解成 Agent 的「运行外壳」：

模型	Harness
决定下一步想做什么	提供当前能做什么，并控制这一步怎样执行

2.6.2 它与 pi 和常见 Agent Harness 有什么区别？

比较维度	pi 0.83.0	DeepSeek Harness	常见的应用级 Agent 框架
主要关注点	轻量、可组合的模型调用与 Agent Loop	插件优先的 Agent 运行平台	快速搭建固定 Workflow 或 Agent 应用
工具扩展	registerTool() 注册工具，使用执行前后事件扩展行为	插件通过 ctx.tools Service 注册工具	通常在初始化时传入工具列表或节点
核心 Loop	源码集中，适合直接理解调用和回传顺序	运行能力通过插件与事件协作，强调可替换和装配	Loop 或 Workflow 通常由框架预先定义
依赖处理	扩展使用宿主提供的 API 和上下文	插件声明所需 Service，依赖未就绪时等待加载	多由应用启动代码手工保证初始化顺序
卸载与清理	扩展跟随会话或宿主生命周期管理	注册行为归属插件 Fiber，卸载时自动撤销	经常只考虑启动和注册，较少讨论动态卸载
最值得学习的部分	Agent Loop、Tool Call、Tool Result 和 Hook 顺序	Service、依赖、生命周期和可撤销注册	业务流程建模与快速交付

DeepSeek Harness 的工具插件化设计

2.6.3 四个新概念

概念	先把它理解成什么	解决什么问题
Registry	按名称保存工具定义的索引	Tool Call 到来时，Runtime 到哪里寻找工具
Plugin	向 Harness 贡献一种能力的自包含模块	新增工具或治理能力时，怎样避免修改核心 Loop
Service	Harness 提供给插件使用的具名运行能力	插件怎样获得 Tool Runtime、LLM 或 Agent 能力
Lifecycle	插件从等待、加载、运行到卸载的过程	依赖未就绪怎么办，卸载时由谁清理资源

插件体系在 Registry 外增加了装配和生命周期



真正要学习的四个 Harness 机制

- 1 工具定义与 Agent Loop 分离，新增工具不修改循环；
- 2 插件通过 Service 获取能力，不直接访问 Harness 内部实现；
- 3 注册行为必须可撤销，谁注册就由谁负责清理；
- 4 权限、Sandbox、超时和 Trace 通过稳定扩展点覆盖多类工具。

扩展点与职责分配

扩展位置	适合放什么	为什么不写进每个工具
执行前	权限、审批、Sandbox、参数策略	同一规则需要覆盖多类工具
执行包裹层	超时、重试、Metrics	需要覆盖完整执行时长
执行后	结果转换、敏感信息拦截	业务结果产生后才能判断
最终结果观察	审计、Trace、统计	只观察权威结果，不再改变它

本节小结

完整链路可以收束为四层



其中真正需要 Agent 开发工程师完成的

- 1 把业务能力拆成边界清楚的工具；
- 2 写出模型能理解、程序能校验的协议；
- 3 从可信上下文注入身份和权限；
- 4 区分可重试错误与不可重复的副作用；
- 5 保证非法、越权和未审批调用永远不会执行；
- 6 让每次调用都能通过 Tool Result 和 Trace 被恢复、解释与审计；
- 7 用插件化方式装配工具和治理能力，同时保持 Agent Loop 稳定。

作业

为「订单运营 Agent」补充一个 `cancel_order` 工具

要求：

- 1 Input Schema 包含 `order_id` 和 `reason`，禁止额外字段；
- 2 Output Schema 返回 `order_id`、取消后的状态和业务操作编号；
- 3 Error Schema 复用本节的 `ToolError`；
- 4 权限设为 `order:write`，风险设为 `high`，工具设为非幂等；
- 5 模型能看见 Schema，但用户身份和审批记录不能由模型填写；
- 6 非法参数、无权限或未确认时，业务函数调用次数必须为零；
- 7 超时后不能自动重试取消动作；
- 8 每次成功、拒绝和失败都可以通过 `trace_id` 与 `tool_call_id` 定位。

完成后，学员应能解释

- Q1 Function Calling 为什么是候选动作，而不是函数执行；
- Q2 为什么工具描述、输入 Schema 和权限检查分别解决不同问题；
- Q3 为什么 Tool Result 必须保留 `tool_call_id`；
- Q4 为什么权限信息不能从模型参数中读取；
- Q5 为什么只有幂等工具的临时错误适合 Runtime 重试；
- Q6 `pi` 和 `DeepSeek Harness` 的插件机制怎样避免把治理逻辑写进每个工具。